



Solución Interrogación 2

Esta evaluación consta de 3 preguntas, donde por cada una se recibe una nota entre 1,0 y 7,0. La nota final de la evaluación es el promedio de las tres preguntas.

Pregunta 1

- a) Suponga que tiene dos modelos entrenados, un MLP y un SVM lineal, donde ambos obtiene métricas de rendimiento iguales. ¿Es posible asegurar que ambos modelos aprenden esencialmente lo mismo? Justifique claramente su respuesta y esboce a partir de ella un algoritmo para medir la similitud de aprendizaje entre dos modelos. (1,5 ptos.)

Respuesta: no se puede asegurar que ambos modelos hayan aprendido lo mismo. Aunque el SVM lineal y el MLP tengan un rendimiento similar, podrían estar clasificando de manera muy distinta ejemplos específicos, lo que podría esconderse al momento de calcular las métricas de rendimiento de forma agregada. Esto puede ocurrir incluso si el MLP utiliza una superficie de clasificación similar a la del SVM.

Para estimar qué tan similares son, se puede calcular cuántas veces ambos modelos hacen la misma predicción en un conjunto de datos de validación:

$$\text{Similitud} = \frac{\text{número de ejemplos con misma predicción}}{\text{total de ejemplos}}$$

Si la similitud es alta, es una señal, aunque no una garantía, de que los modelos están tomando decisiones parecidas.

- b) ¿Cómo afecta el valor de la constante C al número de vectores de soporte en un Soft-Margin SVM? ¿Existen casos donde el variar su valor no tenga efecto en los vectores de soporte? (1,5 ptos.)

Respuesta: un valor alto de C reduce la tolerancia a errores, lo que suele disminuir la cantidad de vectores de soporte. Un valor bajo permite más errores, aumentando su número. Sin embargo, si los datos son perfectamente separables, variar C , por encima de cierto umbral, no afecta los vectores de soporte: estos se mantienen fijos sobre el margen.

- c) Se entrena un árbol de decisión profundo, un Random Forest y un modelo de Gradient Boosting sobre el mismo conjunto de entrenamiento. Luego, se prueba cada uno sobre ejemplos con ruido agregado en las *features*. Compare la robustez de los tres modelos frente al ruido. ¿Cuál esperaría que se vea más afectado y cuál menos? Justifique su respuesta explicando el comportamiento de cada modelo. (1,5 ptos.)

Respuesta: el modelo más robusto al ruido suele ser el Random Forest, ya que promedia las predicciones de muchos árboles entrenados con subconjuntos aleatorios de datos y *features*, lo que reduce la varianza y mitiga el efecto del ruido en variables específicas. El ensamble construido con Gradient Boosting tiende a ser más sensible al

ruido, ya que corrige errores en forma secuencial, lo que puede llevar a sobreajustar patrones espurios si hay ruido en los datos. El árbol de decisión individual es el más frágil, ya que toma decisiones duras y puede sobreajustarse fácilmente al ruido presente en los datos de entrenamiento.

- d) Dos redes neuronales con la misma arquitectura y función de activación son entrenadas sobre el mismo conjunto de datos, pero con inicializaciones aleatorias distintas. Al evaluar ambas redes, se obtiene un rendimiento similar en validación, pero las predicciones sobre los mismos ejemplos no siempre coinciden. ¿Cómo se explica este fenómeno desde el punto de vista de la optimización? ¿Qué implica sobre el espacio de soluciones en redes neuronales y su estabilidad? **(1,5 pts.)**

Respuesta: las redes neuronales con la misma arquitectura pueden converger a soluciones diferentes debido a la no convexidad del espacio de pérdida. La inicialización aleatoria de los pesos puede llevar a mínimos locales distintos o a regiones planas del paisaje de optimización con valores de pérdida similares pero con representaciones internas y decisiones distintas. El hecho de que ambas redes tengan rendimiento similar pero generen predicciones diferentes implica que hay múltiples soluciones igualmente buenas en términos de error promedio, pero no equivalentes a nivel de decisión puntual. Esto revela que las redes neuronales pueden ser inestables en su comportamiento predictivo, incluso si generalizan bien en promedio.

Pregunta 2

- a) Considere dos agentes que resuelven un mismo proceso de decisión de markoviano (MDP) utilizando Q-Learning. Ambos utilizan los mismos valores para la tasa de aprendizaje α y el muestreo de la siguiente acción ε -greedy, pero difieren en su función de recompensa: uno de los agentes, Agente A, utiliza la recompensa original del entorno, mientras que el otro, Agente B, utiliza una versión transformada $R'(s) = a \cdot R(s) + b$, con $a > 0$ y $b \in \mathbb{R}$.

¿Cómo afectaría esta transformación de la recompensa al comportamiento de ambos agentes? ¿Podría alguno de ellos aprender una política diferente? Justifique analíticamente. **(3,0 pts.)**

Respuesta: la transformación aplicada por el Agente B corresponde a una transformación afín de la función de recompensa: $R'(s) = a \cdot R(s) + b$, con $a > 0$ y $b \in \mathbb{R}$. Esta transformación no altera la dinámica del MDP (estados, acciones, ni transiciones), sólo modifica los valores numéricos de las recompensas.

En Q-Learning, la actualización de los valores Q sigue la regla:

$$Q_{k+1}(s, a) \leftarrow Q_k(s, a) + \alpha \left(R(s') + \gamma \max_{a'} Q_k(s', a') - Q_k(s, a) \right)$$

Aplicando una transformación afín a la recompensa, se puede demostrar que:

- Si $a > 0$, el orden relativo de las políticas no cambia. Es decir, la política óptima que maximiza el retorno esperado bajo R también lo es bajo R' , ya que las transformaciones afines con $a > 0$ preservan el orden de preferencia entre acciones.
- Los valores aprendidos por cada agente ($Q(s, a)$) sí difieren, ya que son afectados por la escala (a) y el desplazamiento (b). Sin embargo, esto no impacta la política si se escoge la acción óptima con $\arg \max_a Q(s, a)$.

En conclusión, **ambos agentes aprenderán la misma política óptima**, aunque con valores Q distintos. Por tanto, la transformación de la recompensa afecta los valores numéricos del aprendizaje pero no el comportamiento final del agente.

- b) Considere un conjunto de vectores $\{x_1, x_2, \dots, x_N\} \subset \mathbb{R}^d$ que representan ejemplos de una única clase conocida

como normal o regular. Se desea construir un modelo que, dado un nuevo vector x , determine si este pertenece a la misma distribución o si debe ser considerado una anomalía. Proponga una formulación matemática inspirada en SVM que permita encontrar una región de forma esférica que contenga la mayor parte de los datos. Defina las variables, la función objetivo y cómo modelaría la posibilidad de que algunos puntos queden fuera de dicha región. **(3,0 ptos.)**

Respuesta: la idea es encontrar una **esfera de radio mínimo** que contenga la mayoría de los puntos. Esta esfera se define por un centro $a \in \mathbb{R}^d$ y un radio $R > 0$. Para permitir que algunos puntos queden fuera (anomalías potenciales), se introducen variables de holgura $\xi_i \geq 0$.

La formulación del problema es la siguiente:

$$\underset{R, a, \{\xi_i\}}{\operatorname{argmin}} \quad R^2 + C \sum_{i=1}^N \xi_i$$

sujeto a:

$$\begin{aligned} \|x_i - a\|^2 &\leq R^2 + \xi_i, \quad \forall i \in \{1, \dots, N\} \\ \xi_i &\geq 0 \end{aligned}$$

El objetivo es minimizar el radio de la esfera, penalizando puntos que queden fuera mediante las variables ξ_i . El parámetro C controla el compromiso entre el tamaño de la esfera y la cantidad de puntos permitidos fuera de ella (es decir, la tolerancia al error o anomalías).

Pregunta 3

Se desea construir una red neuronal que reproduzca el comportamiento de un árbol de decisión entrenado previamente. Es decir, se busca que la red imite la función de clasificación del árbol, no solo sus predicciones, usando capas y funciones de activación adecuadas para redes neuronales.

- a) Explique cómo se podría representar cada nodo de decisión del árbol dentro de una red neuronal. ¿Qué estructura debe tener la red para simular las decisiones jerárquicas del árbol? ¿Qué tipo de funciones de activación serían apropiadas? **(2,0 ptos.)**

Respuesta: cada nodo interno de un árbol binario evalúa una condición del tipo “¿la característica x_j es menor que un umbral t ?”. Esto se puede aproximar con una neurona que tome como entrada la componente x_j y aplique una función de activación que genere un valor cercano a 0 o 1 según el resultado de la comparación. Las funciones más comunes para esto son la sigmoide y la ReLU, aunque ninguna produce cortes duros como los árboles.

Para simular la jerarquía del árbol, se puede construir una red donde cada capa represente una profundidad del árbol. Las combinaciones de activaciones en cada ruta determinan qué “hoja” se activa. Las hojas del árbol se representan como neuronas en la capa de salida, donde cada una corresponde a una predicción. En resumen, la red debe imitar las rutas del árbol mediante composiciones de funciones no lineales.

- b) ¿Qué dificultades surgen al intentar entrenar directamente una red neuronal para que aprenda a imitar un árbol, sin conocer su estructura previa? ¿Qué características del aprendizaje de redes podrían impedir una correspondencia exacta con el árbol? **(2,0 ptos.)**

Respuesta: entrenar directamente una red neuronal para imitar un árbol sin conocer su estructura presenta varias dificultades:

- Las redes aprenden representaciones distribuidas y suaves, mientras que los árboles hacen decisiones locales y abruptas.
- El espacio de hipótesis de una red es continuo y redundante, mientras que el de un árbol es discreto y jerárquico.
- Los árboles realizan divisiones duras del espacio de entrada; en cambio, las redes generan transiciones suaves.

Esto puede llevar a que la red aprenda un comportamiento funcionalmente parecido (en términos de predicción), pero muy distinto en su estructura interna. Además, sin una pérdida explícita que induzca una estructura tipo árbol, la red podría no converger a una solución interpretable.

- c) Suponga ahora que usted toma un enfoque distinto y entrena un MLP tradicional usando como etiqueta de los datos las predicciones generadas por un árbol, donde este árbol sí fue entrenado con las etiquetas reales de los datos. ¿Qué propiedades podría adquirir la red que no tiene el árbol? ¿Qué ventajas y desventajas tiene este procedimiento? **(2,0 ptos.)**

Respuesta: si se entrena una red neuronal sobre las predicciones del árbol (en vez de las etiquetas reales), se obtiene una red que aproxima la función de decisión del árbol. Esto se conoce como *knowledge distillation*. Algunas ventajas de este enfoque son:

- La red puede ser más compacta, diferenciable y compatible con técnicas modernas de entrenamiento.
- Puede generalizar mejor, al suavizar fronteras abruptas que el árbol haya aprendido.

Entre las desventajas están:

- La red hereda los errores o sesgos del árbol.
- Si el árbol es muy complejo o sobreajustado, la red podría necesitar muchos parámetros para replicarlo.
- La calidad del entrenamiento depende de cómo se representen las salidas del árbol (clases duras vs. probabilidades).

- d) **Bonus:** ¿cómo podría adaptarse la estrategia de representación de la pregunta a) para modelar un Random Forest? **(1 pto.)**

Respuesta: la estrategia de la pregunta a), en la que se representa un árbol binario como una red neuronal con capas que modelan las decisiones jerárquicas y hojas como neuronas de salida, puede adaptarse a un Random Forest construyendo una subred independiente para cada árbol del ensamble.

Cada subred tendría la misma estructura que se usaría para representar un solo árbol, con sus propias neuronas y pesos. Luego, las salidas de estas subredes (correspondientes a las predicciones de cada árbol) se podrían combinar en una capa final mediante un promedio (en el caso de regresión) o una votación suavizada (por ejemplo, usando una capa densa con pesos fijos o entrenables que simulan una agregación).

Esta arquitectura permite que la red neuronal represente de forma explícita la estructura del bosque y se puede inicializar directamente a partir del modelo entrenado, permitiendo además fine-tuning posterior o integración en arquitecturas más grandes.